

How to do a hitscan ?



What is the Out Keyword in Unity?

The out keyword in Unity is used to return extra information from a function.

Normally, when calling a function, you pass data into it via its parameters and get information back via its return type.

It's likely that you'll have already written functions with a **void** return type, meaning they don't return any information.

Other functions, on the other hand, may specify what type of data is returned from the function when it runs.

```
bool IsAboveTen(float num, out
string result)
{
    if (num > 10)
    {
        result = "The number was
more than 10!";
        return true;
    }
    else
    {
        result = "The number was
not more than 10!";
        return false;
    }
}
```

Raycasting



Raycast

The Raycast function is extremely useful for creating connections between objects in Unity.

For example, if you want to select an object just by looking at it or clicking on it.

Or if you want to shoot an object with a weapon or obstruct a line of sight.

Or perhaps if you need to check how high the player is from the ground, or what it is they're standing on.

That's where Raycasts can help.



What is the Raycast function ?

Raycast in Unity is a Physics function that projects a Ray into the scene, returning a boolean value if a target was successfully hit.

When this happens, information about the hit, such as the distance, position or a reference to the object's Transform, can be stored in a Raycast Hit variable for further use.

Raycast are extremely useful for getting information about other objects, performing ground checks or, generally, doing anything that involves a line of sight connection between two objects.

How to create a Ray

A **Ray** is simply a data struct in Unity that represents a point of **origin** and a **direction** for the Ray to travel.

The **origin** is the position in the world that the Ray will start from and takes a **Vector3** value.

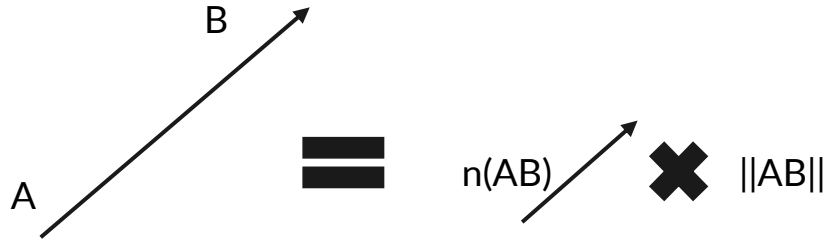
While the **direction** property takes a trajectory in the form of a normalised **Vector3**.



Normalizing a vector

Normalising a vector limits its magnitude, which is the length of the Vector, to 1, which is useful for representing a consistent direction. A normalized vector **maintains its direction** but its **Length becomes 1**. The resulting vector is often called a **unit vector**.

A vector is normalized by dividing the vector by its own Length.


$$\vec{AB} = \frac{\vec{AB}}{\|\vec{AB}\|}$$



Creating a new ray

There are a few ways you can create a new Ray.

```
Ray ray = new Ray(transform.position, transform.forward);
```

Or, alternatively, you can use one of the helper functions to create a Ray automatically, such as a Ray that starts at the

```
Ray ray = Camera.main.ViewportPointToRay(new Vector3 (0.5f, 0.5f, 0));
```



Creating a new ray

Or from the Mouse Position on the screen,

```
// Creates a Ray from the mouse position  
Ray ray = Camera.main.ScreenPointToRay(Input.mousePosition);
```

These helper functions make it easy to create a Ray from an existing point in the world.

However, while you can cache a reference to the Ray you create, you'll still need to update it **every time you use it.**



Updating a ray

So, for example, if your Ray's origin and direction are going to be different every frame, you'll need

```
Ray ray;
void Update()
{
    ray = transform.position, transform.forward;
}
```

Otherwise, when you come to use the Ray, it won't be in the right position.

How to visualise Raycast in the scene

The **Debug Class** includes a function, **Draw Ray**, that allows you to render a Ray in the world.

```
Debug.DrawRay(Vector3 origin, Vector3 direction);
```



*Using **Debug.DrawRay**, you can check if your Rays are going where they're meant to.*



How to use the Raycast Hit variable

The Raycast Hit variable is a data struct that stores information about Ray's collision.

It records information like where the hit happened in the world, what object did the Ray collide with and how far away it was from the origin of the Ray.

To use it, all you need to do is declare it:

```
// Container for hit data  
  
RaycastHit hitData;
```



How to use the Raycast Hit variable :

Then, later, when you use Raycast to fire a Ray into the scene, you'll have a place to store any information retrieved from the hit.

Using RaycastHit.point:

```
Vector3 hitPosition = hitData.point;
```

Or the distance from the Ray's origin to the point of impact using RaycastHit.distance:

```
float hitDistance = hitData.distance;
```



How to use the Raycast Hit variable :

You can also get information about the **Collider** that was hit, such as its **Tag**.

```
// Reads the Collider tag  
  
string tag = hitData.collider.tag;
```

Or, you could even use Raycast Hit to get a reference to an object's **Transform**:

```
// Gets a Game Object reference from its Transform  
  
GameObject hitObject = hitData.transform.gameObject;
```



How to use the Raycast function

The Raycast function in Unity allows you to check if a Ray collides with another object in the scene, saving the hit data to a **Raycast Hit** variable if it does.

There are several different versions of Raycast, each from different classes, that can be used in different ways, however, one of the most common ways to use Raycast is using the **Physics Class**, which returns a boolean true or false, depending on if the Ray hit anything.



Example of a using a Raycast Function

```
void FireRay()
{
    Ray ray = new Ray(transform.position, transform.forward);

    RaycastHit hitData;

    Physics.Raycast(ray, out hitData);
}
```

This will fire the ray that you've created into the scene and save information about anything that's hit to the Raycast Hit variable you declared.

This works because of the **Out Keyword** in the function.



Understanding the Physics.Raycast

Raycast takes an **out Raycast Hit** argument, meaning that a **Raycast Hit** variable will need to be specified when you call it.

Then when the function runs, **Raycast Hit data** will be stored in the variable you specified, allowing you to access it. The **out** keyword allows you to save information in addition to the return type of Raycast.

So what is the return type? Physics.Raycast has a **bool** return type, which returns true if the Ray hits a target.

This means that you can place the Raycast function in an **if condition**.



Using raycast in an If condition

Using Raycast in this way means that code inside the if condition will only run if the Ray actually hits something.

```
void Update()
{
    Ray ray = new Ray(transform.position, transform.forward);
    RaycastHit hitData;
    if (Physics.Raycast(ray, out hitData))
    {
        // The Ray hit something!
    }
}
```



The different features of the Raycast function

There are many different versions of the standard Raycast function and each one offers slightly different functionality.

For example, some versions of Raycast only take a few parameters and perform a simple function in return, while others accept more complex sets of data and do more with it.





Limit the Raycast range by entering a max distance

Most of the overload methods available for `Physics.Raycast` also allow you to specify a max distance as a float value.

```
Ray ray = new Ray(transform.position, transform.forward);  
if (Physics.Raycast(ray, 10))  
{  
    // Hit Something closer than 10 units away  
}
```



How to use Layer Masks with Raycasts

One of the most helpful features of the Raycast function is the ability to filter out colliders depending on their layer.

This allows you to completely ignore objects on layers that aren't intended to collide with Raycasts.

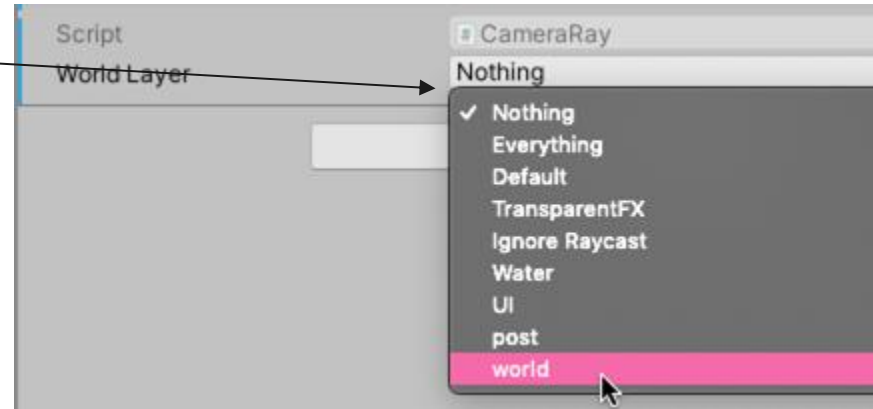
This can extremely useful when you're working with a large scene, with many different types of objects and colliders and you only want to check for collisions against some of them.

How to use Layer Masks with Raycasts

For example, let's say that all of the environment objects in my scene are on the same layer, the world layer, and that I only want to detect collisions on that layer.

All I need to do is declare a public Layer Mask variable

```
public LayerMask worldLayer;
```





How to use Layer Masks with Raycasts

And then, in my script, pass the Layer Mask variable into the Raycast function.

```
public LayerMask worldLayer;
void FireLaser()
{
    Ray ray = Camera.main.ViewportPointToRay(new Vector3(0.5f, 0.5f, 0));
    if (Physics.Raycast(ray, 10, worldLayer))
    {
        Debug.Log("You hit a wall, good job!");
    }
}
```



Detect Raycast on all layers except for one

This can be done with the bitwise NOT operator. The bitwise NOT operation use the tilde

```
public LayerMask worldLayer;
void FireLaser()
{
    Ray ray = new Ray(transform.position, transform.forward);
    if (Physics.Raycast(ray, 10, ~worldLayer))
    {
        // Something other than the world was hit!
    }
}
```



How to ignore trigger colliders when using Raycast

By default, Raycast will hit triggers, meaning that if a Raycast impacts a trigger collider, it'll behave in the same way as hitting any other collider.

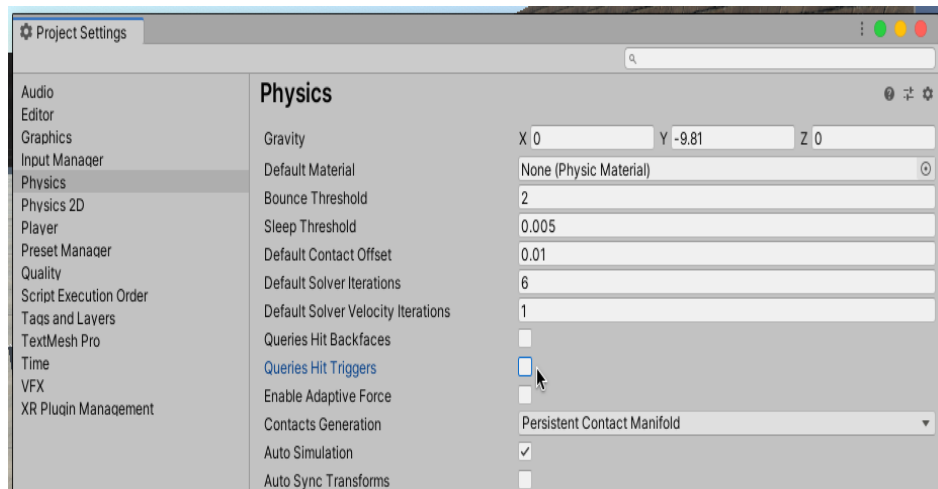
It's possible to change this behaviour, either globally via a project setting or on a per Raycast basis.

How to disable all Raycast Trigger collisions

The easiest way to disable all Raycast trigger collisions is to simply turn them off.

Open **Project Settings**, select the **Physics** menu (or Physics 2D menu if you're working in 2D) and uncheck **Queries Hit Triggers**.

Now, by default, Raycasts will ignore all trigger collisions.





How to disable individual Raycast Trigger collisions

```
void FireLaser()
{
    Ray ray = Camera.main.ViewportPointToRay(new Vector3(0.5f, 0.5f, 0));
    if (Physics.Raycast(ray, 10, worldLayer, QueryTriggerInteraction.Ignore))
    {
        // Whatever you hit, it wasn't a trigger
    }
}
```



How to hit multiple objects using Raycast

While Raycast is useful for identifying or colliding with single objects, sometimes you might want to use Raycast to detect collisions against multiple objects using the same Ray.

Raycast All works a lot like Raycast, except that instead of hitting and returning only one object it can return multiple objects, storing them in an array.

To use Raycast All, you'll need to declare an array of Raycast Hit variables and assign them when you call the Raycast All function.



How to hit multiple objects using Raycast

Raycast All works a lot like Raycast, except that instead of hitting and returning only one object it can return multiple objects, storing them in an array.

To use **Raycast All**, you'll need to declare an array of **Raycast Hit** variables and assign them when

```
public RaycastHit[] hits;
void Update()
{
    Ray ray = new Ray(transform.position, transform.forward);
    hits = Physics.RaycastAll(ray);
}
```



How to hit multiple objects using Raycast

Raycast All is useful for getting information about multiple objects using a single Ray.

For example:

```
int numObjectsHit = hits.Length;
```



How to hit multiple objects using Raycast

Or you could destroy all of the objects in a Ray's path :

```
public RaycastHit[] hits;
void FireLaser()
{
    Ray ray = new Ray(transform.position, transform.forward);
    hits = Physics.RaycastAll(ray);
    foreach(RaycastHit obj in hits)
    {
        Destroy(obj.transform.gameObject);
    }
}
```